Practical-4

Problem Definition

Implement and evaluate two different data integration pipelines using identical retail transaction data. The first pipeline follows the ETL (Extract, Transform, Load) architecture, where raw data is extracted from local storage, transformed in application memory using Pandas, and then loaded into a DuckDB analytical database.

The second pipeline follows the ELT (Extract, Load, Transform) architecture, where raw transaction data is first loaded directly into a DuckDB staging table and transformations are then performed using the DuckDB SQL engine.

The practical compares both architectures based on execution time, processing efficiency, data lineage, transformation recovery, resource utilization, and maintenance requirements.

Dataset

The dataset consists of high-volume, un-normalized retail sales transaction records.

The raw dataset contains 100,000 retail transaction records with fields such as:

- Transaction ID
- Store Code
- Product
- Category
- Quantity
- Price
- Sale Date

The dataset intentionally contains data-quality inconsistencies such as:

- Leading and trailing white spaces
- Inconsistent category capitalization
- Legacy store codes
- Raw numeric values
- Unstandardized product names

ETL Architecture

The ETL pipeline follows:

Raw CSV File

↓

Extract

↓

Pandas DataFrame

↓

Transform

↓

Cleaned Data

↓

DuckDB

↓

Final Table

ELT Architecture

The ELT pipeline follows:

Raw CSV File

↓

Extract

↓

DuckDB Staging Table

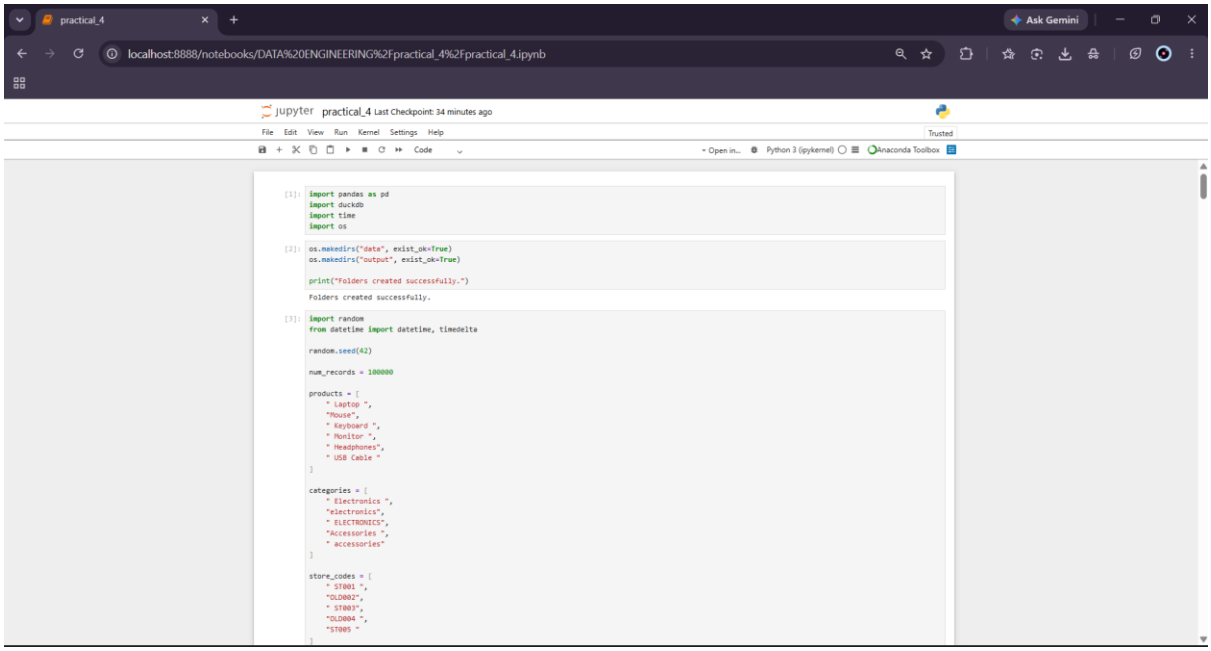
↓

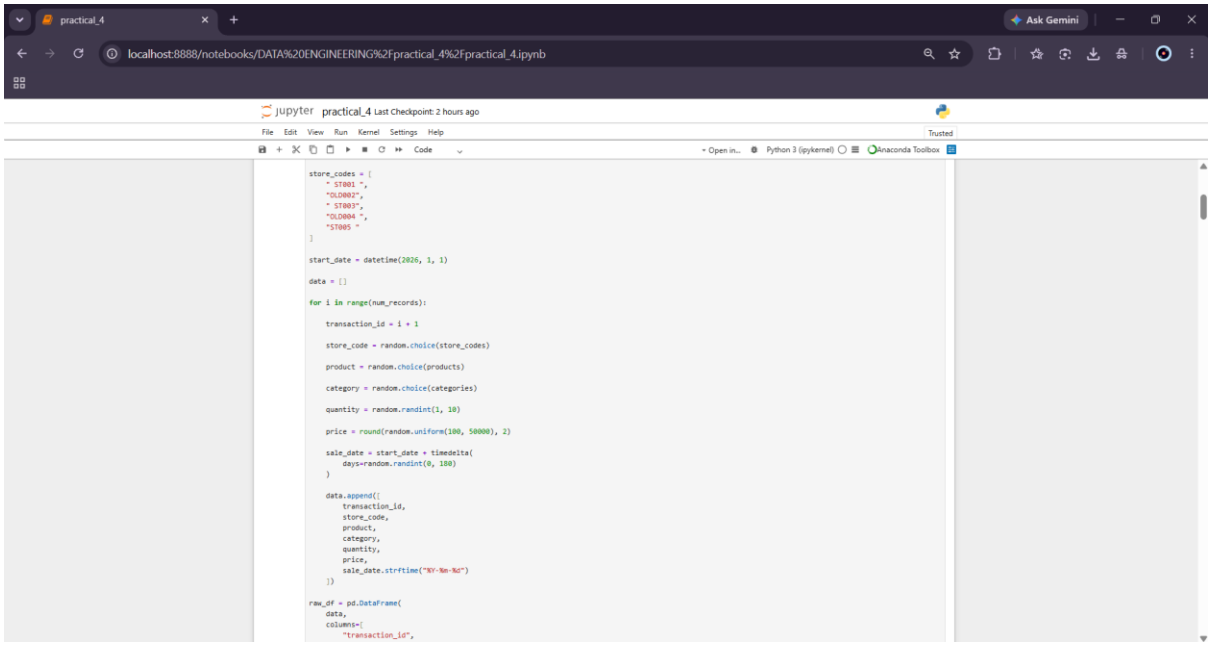
Transform using SQL

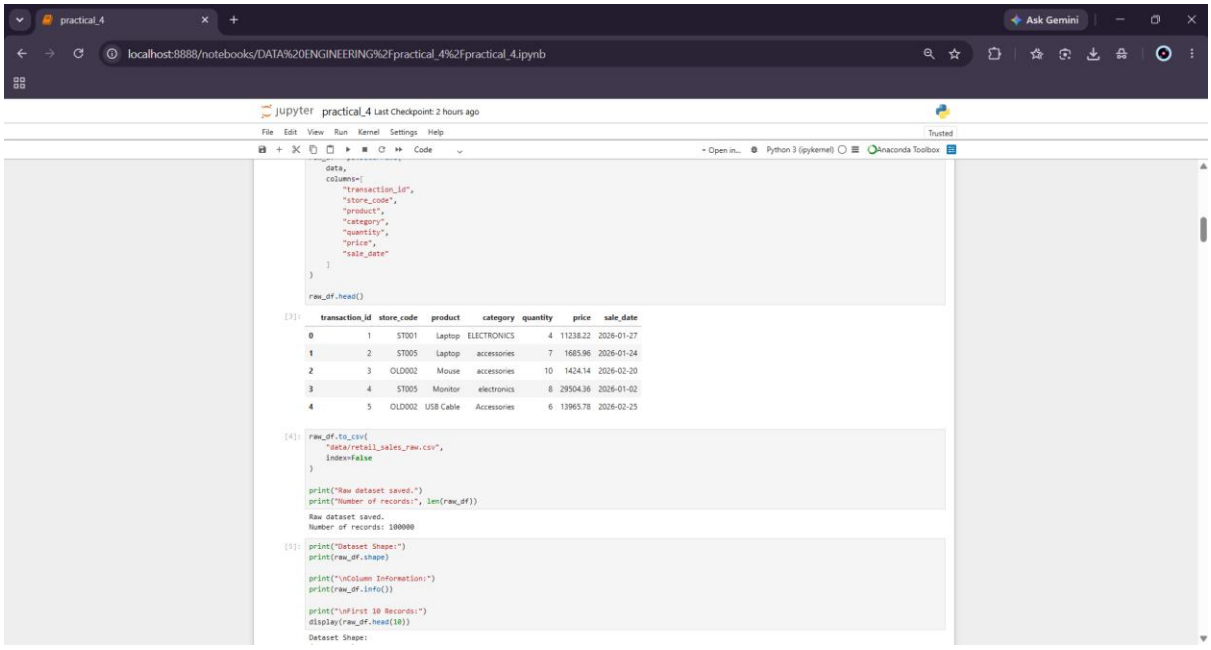
↓

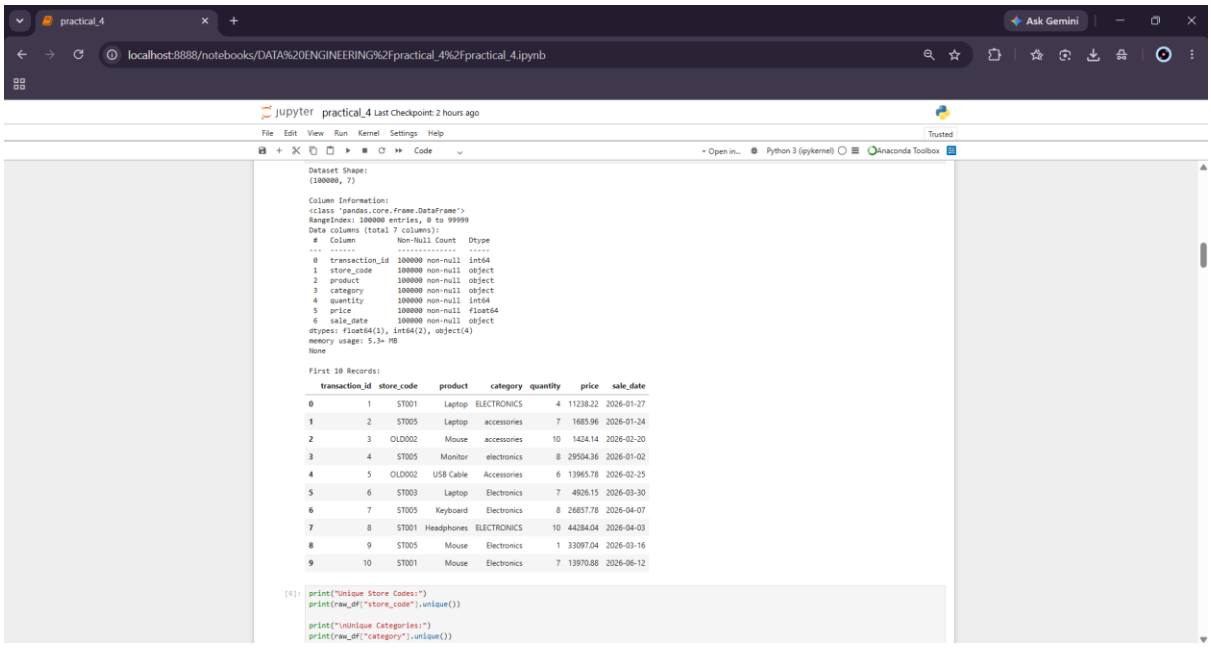
Final DuckDB Table

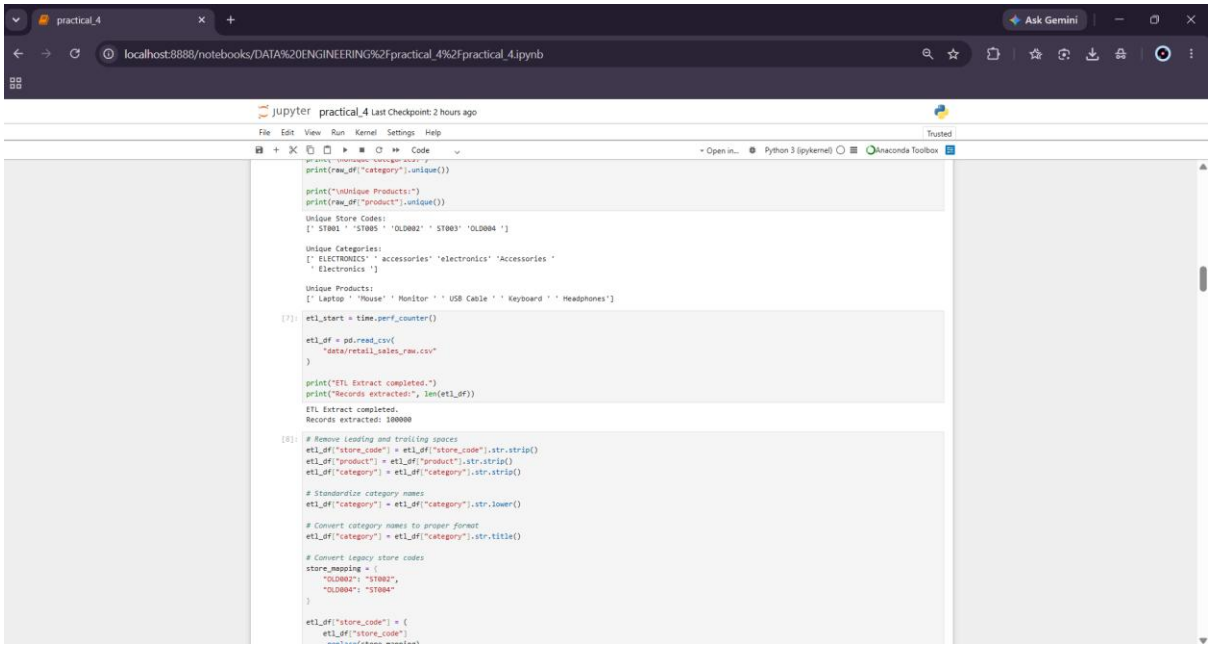
OUTPUT/SCREENSHOTS

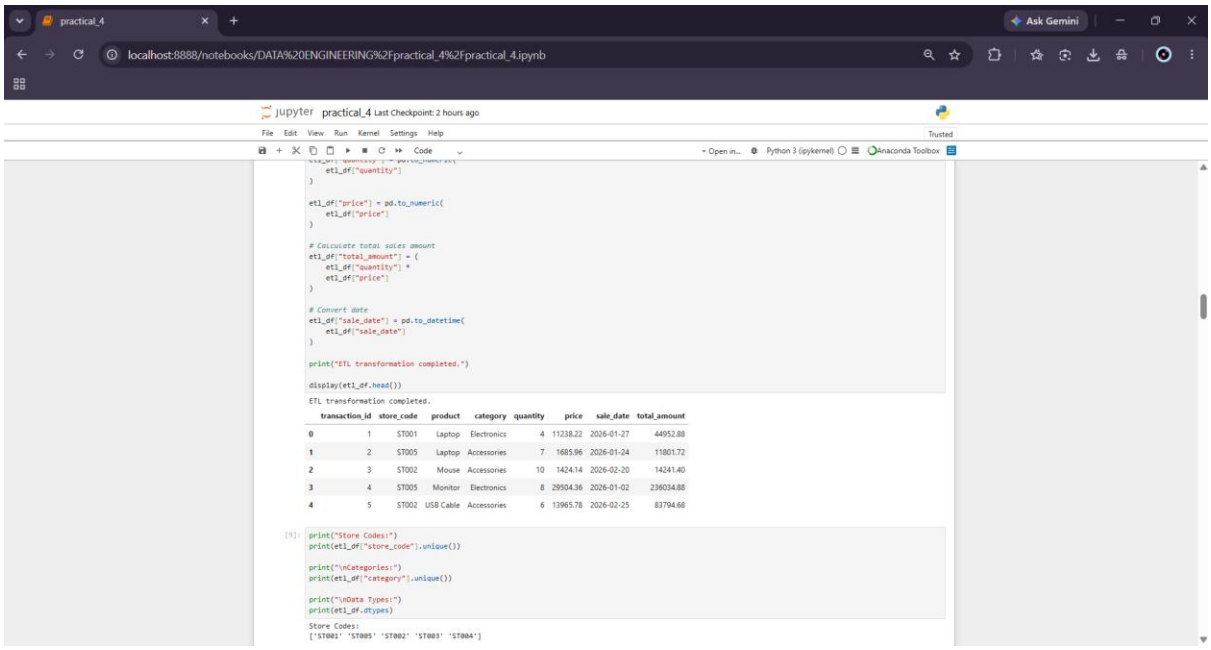


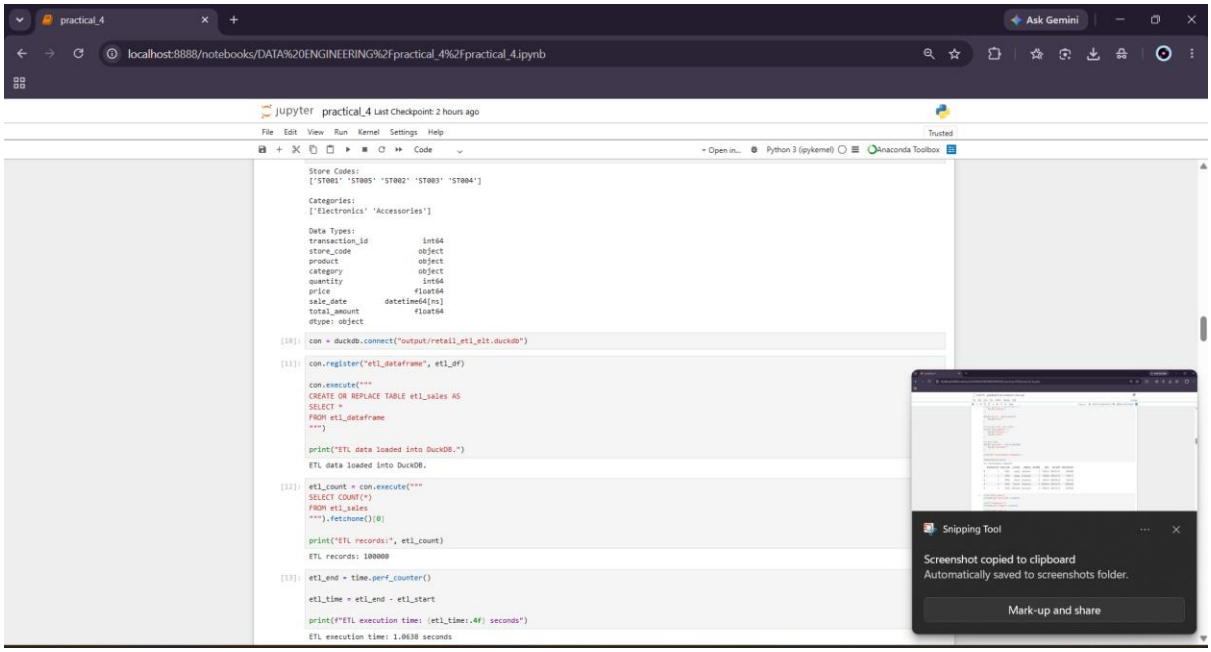


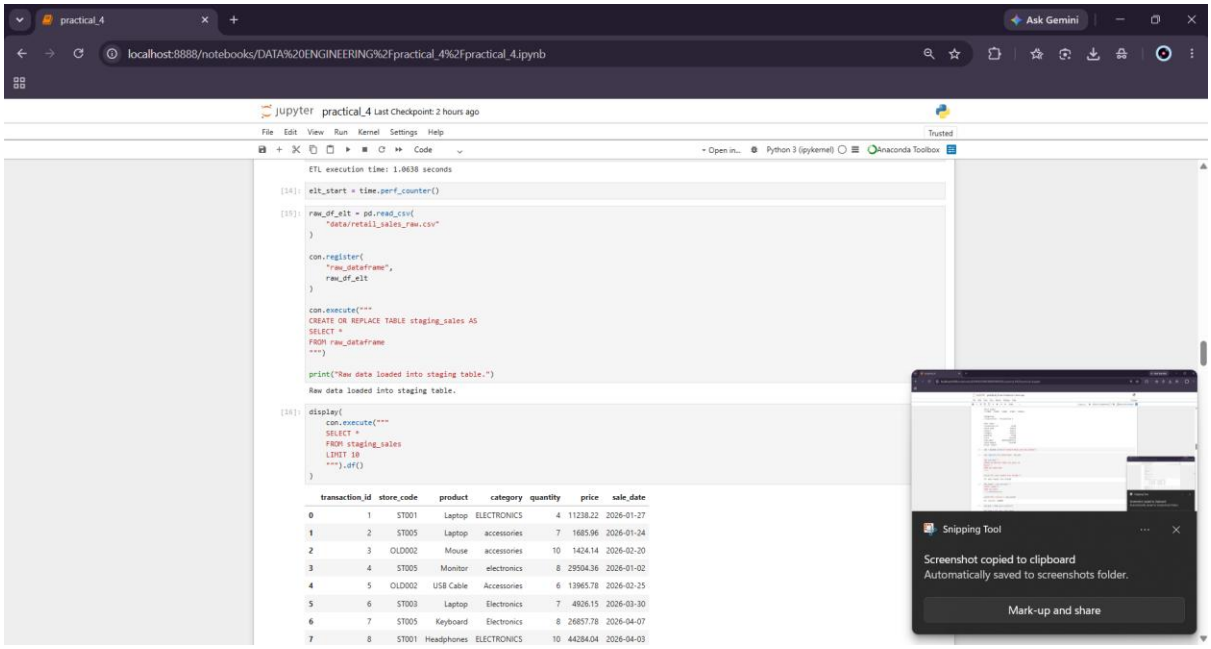

practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
data,
columns=[
"transaction_id",
"store_code",
"product",
"category",
"quantity",
"price",
"sale_date"
]
)
raw_df.head()
[3]:
transaction_id store_code product category quantity price sale_date
0 1 ST001 Laptop ELECTRONICS 4 11238.22 2026-01-27
1 2 ST005 Laptop accessories 7 1685.96 2026-01-24
2 3 OLD002 Mouse accessories 10 1424.14 2026-02-20
3 4 ST005 Monitor electronics 8 29504.36 2026-01-02
4 5 OLD002 USB Cable Accessories 6 13965.78 2026-02-25
[4]:
raw_df.to_csv(
"data/retail_sales_raw.csv",
index=False
)
print("Raw dataset saved.")
print("Number of records:", len(raw_df))
Raw dataset saved.
Number of records: 100000
[5]:
print("Dataset Shape:")
print(raw_df.shape)
print("\nColumn Information:")
print(raw_df.info())
print("\nFirst 10 Records:")
display(raw_df.head(10))
Dataset Shape:

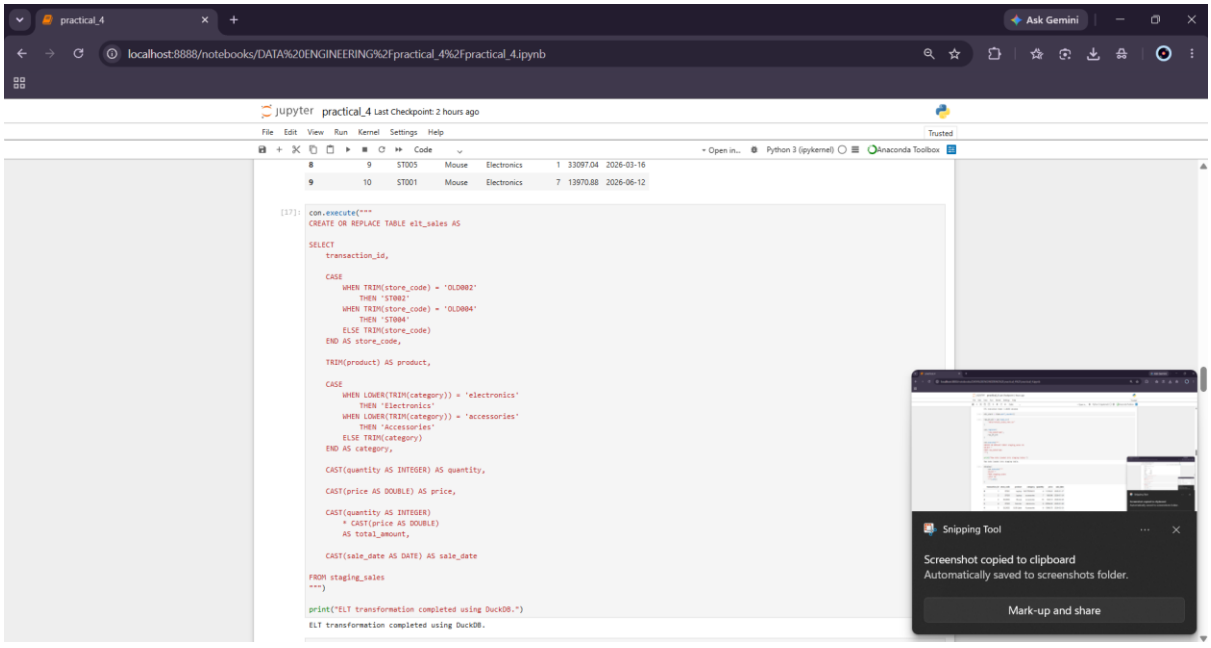

practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
Dataset Shape:
(100000, 7)
Column Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 7 columns):
Column Non-Null Count Dtype
0 transaction_id 100000 non-null int64
1 store_code 100000 non-null object
2 product 100000 non-null object
3 category 100000 non-null object
4 quantity 100000 non-null int64
5 price 100000 non-null float64
6 sale_date 100000 non-null object
dtypes: float64(1), int64(2), object(4)
memory usage: 5.3+ MB
None
First 10 Records:
transaction_id store_code product category quantity price sale_date
0 1 ST001 Laptop ELECTRONICS 4 11238.22 2026-01-27
1 2 ST005 Laptop accessories 7 1685.96 2026-01-24
2 3 OLD002 Mouse accessories 10 1424.14 2026-02-20
3 4 ST005 Monitor electronics 8 29504.36 2026-01-02
4 5 OLD002 USB Cable Accessories 6 13965.78 2026-02-25
5 6 ST003 Laptop Electronics 7 4926.15 2026-03-30
6 7 ST005 Keyboard Electronics 8 26857.78 2026-04-07
7 8 ST001 Headphones ELECTRONICS 10 44284.04 2026-04-03
8 9 ST005 Mouse Electronics 1 33097.04 2026-03-16
9 10 ST001 Mouse Electronics 7 13970.88 2026-06-12
[6]:
print("Unique Store Codes:")
print(raw_df["store_code"].unique())
print("\nUnique Categories:")
print(raw_df["category"].unique())

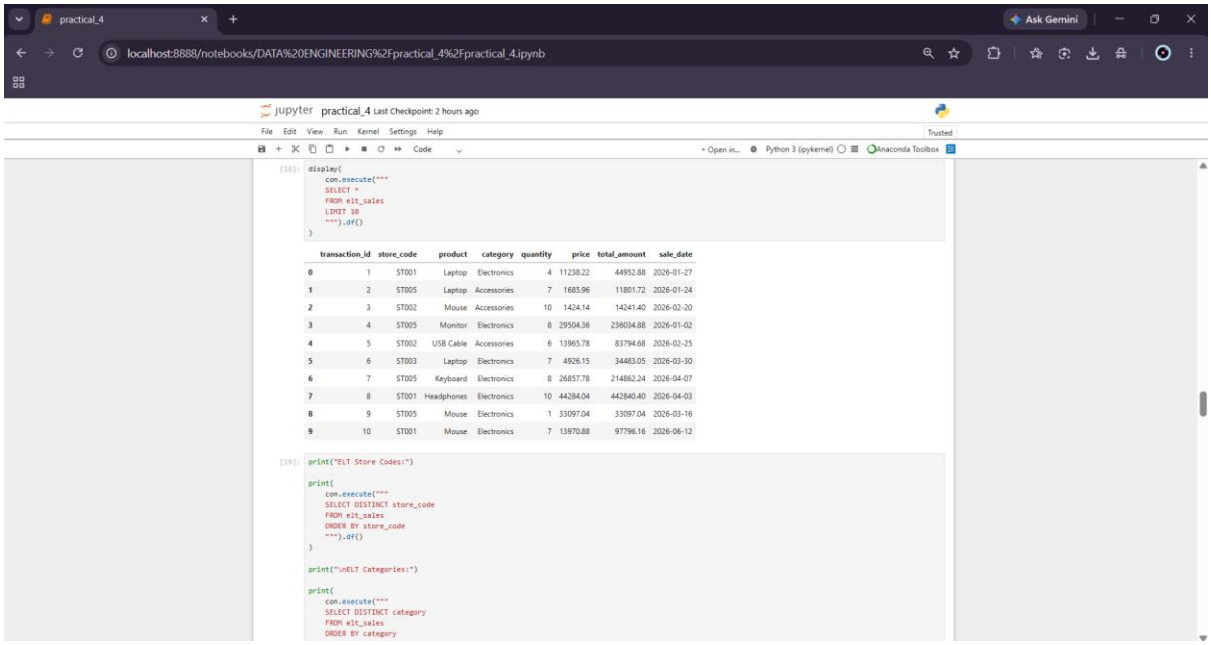

practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
print(raw_df["category"].unique())
print("\nUnique Products:")
print(raw_df["product"].unique())
Unique Store Codes:
[' ST001 ' 'ST005 ' 'OLD002' ' ST003' 'OLD004 ']
Unique Categories:
[' ELECTRONICS' ' accessories' 'electronics' 'Accessories '
' Electronics ']
Unique Products:
[' Laptop ' 'Mouse' ' Monitor ' ' USB Cable ' ' Keyboard ' ' Headphones']
[7]:
etl_start = time.perf_counter()
etl_df = pd.read_csv(
"data/retail_sales_raw.csv"
)
print("ETL Extract completed.")
print("Records extracted:", len(etl_df))
ETL Extract completed.
Records extracted: 100000
[8]:
Remove leading and trailing spaces
etl_df["store_code"] = etl_df["store_code"].str.strip()
etl_df["product"] = etl_df["product"].str.strip()
etl_df["category"] = etl_df["category"].str.strip()
Standardize category names
etl_df["category"] = etl_df["category"].str.lower()
Convert category names to proper format
etl_df["category"] = etl_df["category"].str.title()
Convert legacy store codes
store_mapping = {
"OLD002": "ST002",
"OLD004": "ST004"
}
etl_df["store_code"] = (
etl_df["store_code"]

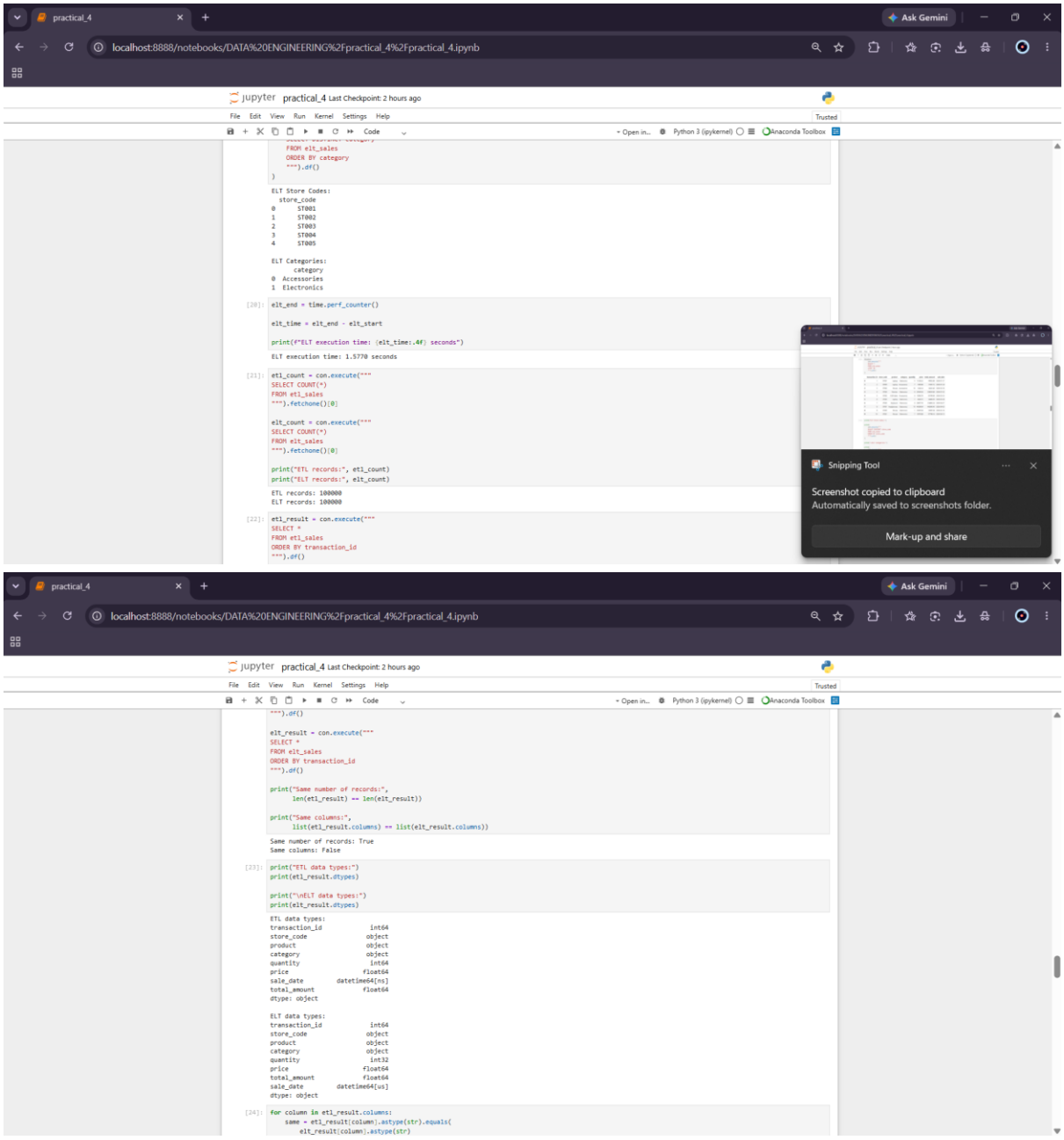

practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
etl_df["quantity"]
)
etl_df["price"] = pd.to_numeric(
etl_df["price"]
)
Calculate total sales amount
etl_df["total_amount"] = (
etl_df["quantity"] *
etl_df["price"]
)
Convert date
etl_df["sale_date"] = pd.to_datetime(
etl_df["sale_date"]
)
print("ETL transformation completed.")
display(etl_df.head())
ETL transformation completed.
transaction_id store_code product category quantity price sale_date total_amount
0 1 ST001 Laptop Electronics 4 11238.22 2026-01-27 44952.88
1 2 ST005 Laptop Accessories 7 1685.96 2026-01-24 11801.72
2 3 ST002 Mouse Accessories 10 1424.14 2026-02-20 14241.40
3 4 ST005 Monitor Electronics 8 29504.36 2026-01-02 236034.88
4 5 ST002 USB Cable Accessories 6 13965.78 2026-02-25 83794.68
[9]:
print("Store Codes:")
print(etl_df["store_code"].unique())
print("\nCategories:")
print(etl_df["category"].unique())
print("\nData Types:")
print(etl_df.dtypes)
Store Codes:
['ST001' 'ST005' 'ST002' 'ST003' 'ST004']


practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
Store Codes:
['ST001' 'ST005' 'ST002' 'ST003' 'ST004']
Categories:
['Electronics' 'Accessories']
Data Types:
transaction_id int64
store_code object
product object
category object
quantity int64
price float64
sale_date datetime64[ns]
total_amount float64
dtype: object
con = duckdb.connect("output/retail_etl_elt.duckdb")
con.register("etl_dataframe", etl_df)
con.execute("""
CREATE OR REPLACE TABLE etl_sales AS
SELECT *
FROM etl_dataframe
""")
print("ETL data loaded into DuckDB.")
ETL data loaded into DuckDB.
etl_count = con.execute("""
SELECT COUNT(*)
FROM etl_sales
""").fetchone()[0]
print("ETL records:", etl_count)
ETL records: 100000
etl_end = time.perf_counter()
etl_time = etl_end - etl_start
print(f"ETL execution time: {etl_time:.4f} seconds")
ETL execution time: 1.0638 seconds
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share

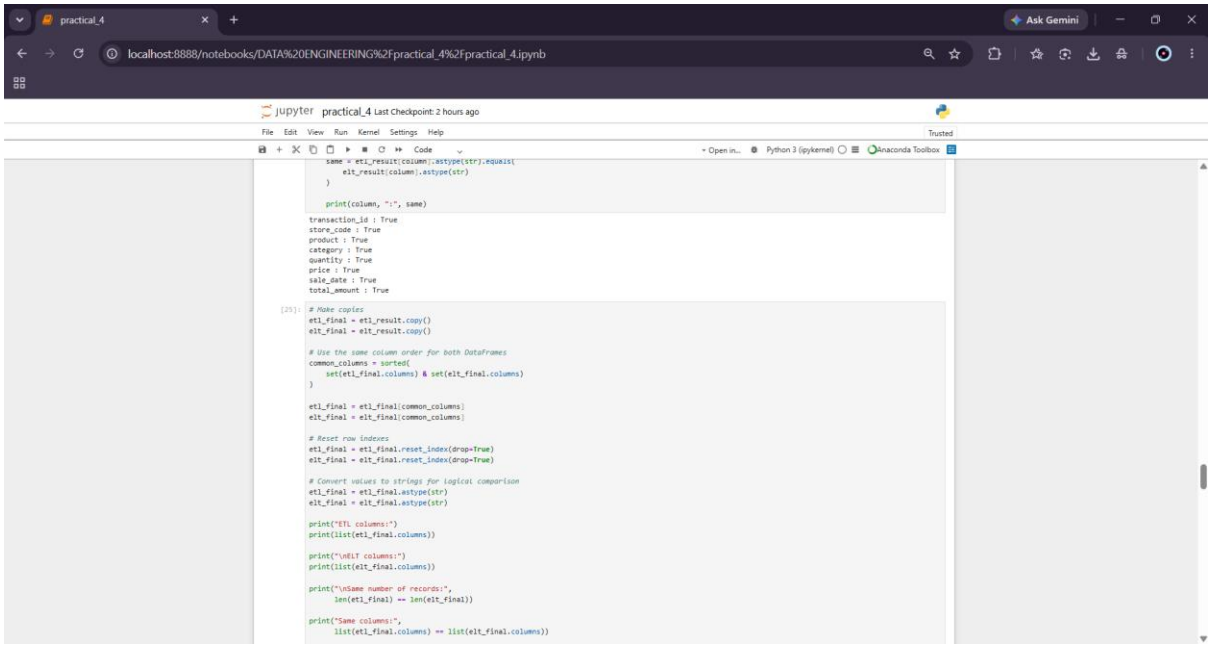

practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
ETL execution time: 1.0638 seconds
elt_start = time.perf_counter()
raw_df_elt = pd.read_csv(
"data/retail_sales_raw.csv"
)
con.register(
"raw_dataframe",
raw_df_elt
)
con.execute("""
CREATE OR REPLACE TABLE staging_sales AS
SELECT *
FROM raw_dataframe
""")
print("Raw data loaded into staging table.")
Raw data loaded into staging table.
display(
con.execute("""
SELECT *
FROM staging_sales
LIMIT 10
""").df()
)
transaction_id store_code product category quantity price sale_date
0 1 ST001 Laptop ELECTRONICS 4 11238.22 2026-01-27
1 2 ST005 Laptop accessories 7 1685.96 2026-01-24
2 3 OLD002 Mouse accessories 10 1424.14 2026-02-20
3 4 ST005 Monitor electronics 8 29504.36 2026-01-02
4 5 OLD002 USB Cable Accessories 6 13965.78 2026-02-25
5 6 ST003 Laptop Electronics 7 4926.15 2026-03-30
6 7 ST005 Keyboard Electronics 8 26857.78 2026-04-07
7 8 ST001 Headphones ELECTRONICS 10 44284.04 2026-04-03
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share

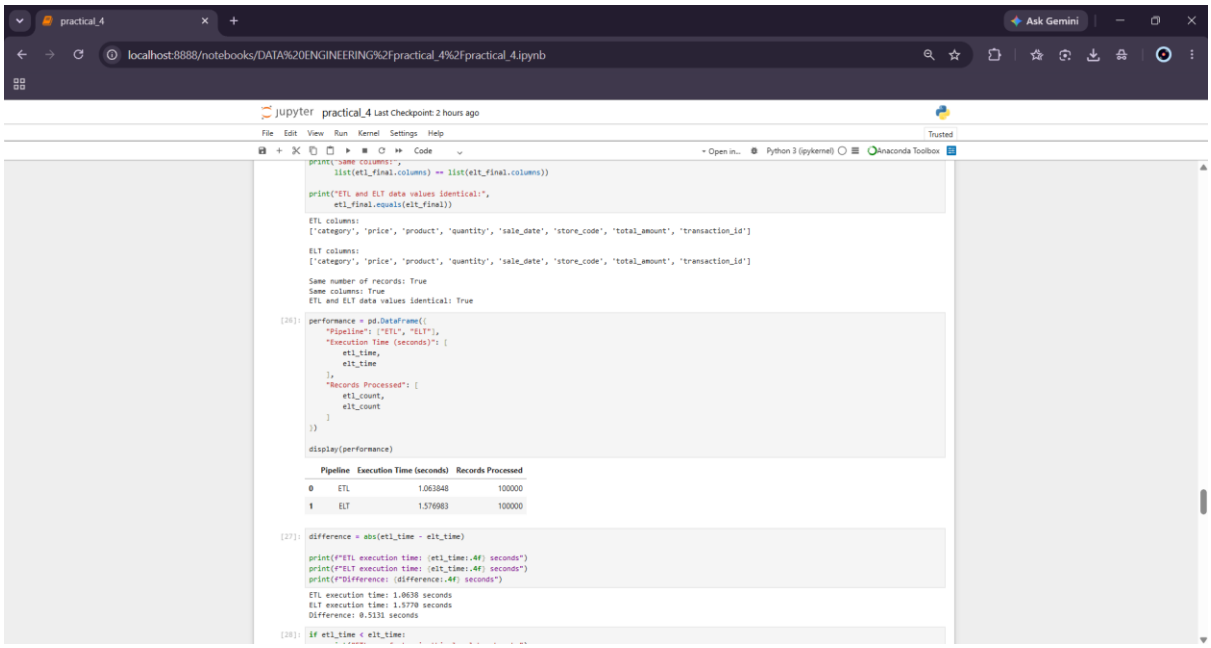

practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
8 9 ST005 Mouse Electronics 1 33097.04 2026-03-16
9 10 ST001 Mouse Electronics 7 13970.88 2026-06-12
[17]:
con.execute("""
CREATE OR REPLACE TABLE elt_sales AS
SELECT
transaction_id,
CASE
WHEN TRIM(store_code) = 'OLD002'
THEN 'ST002'
WHEN TRIM(store_code) = 'OLD004'
THEN 'ST004'
ELSE TRIM(store_code)
END AS store_code,
TRIM(product) AS product,
CASE
WHEN LOWER(TRIM(category)) = 'electronics'
THEN 'Electronics'
WHEN LOWER(TRIM(category)) = 'accessories'
THEN 'Accessories'
ELSE TRIM(category)
END AS category,
CAST(quantity AS INTEGER) AS quantity,
CAST(price AS DOUBLE) AS price,
CAST(quantity AS INTEGER)
* CAST(price AS DOUBLE)
AS total_amount,
CAST(sale_date AS DATE) AS sale_date
FROM staging_sales
""")
print("ELT transformation completed using DuckDB.")
ELT transformation completed using DuckDB.
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share

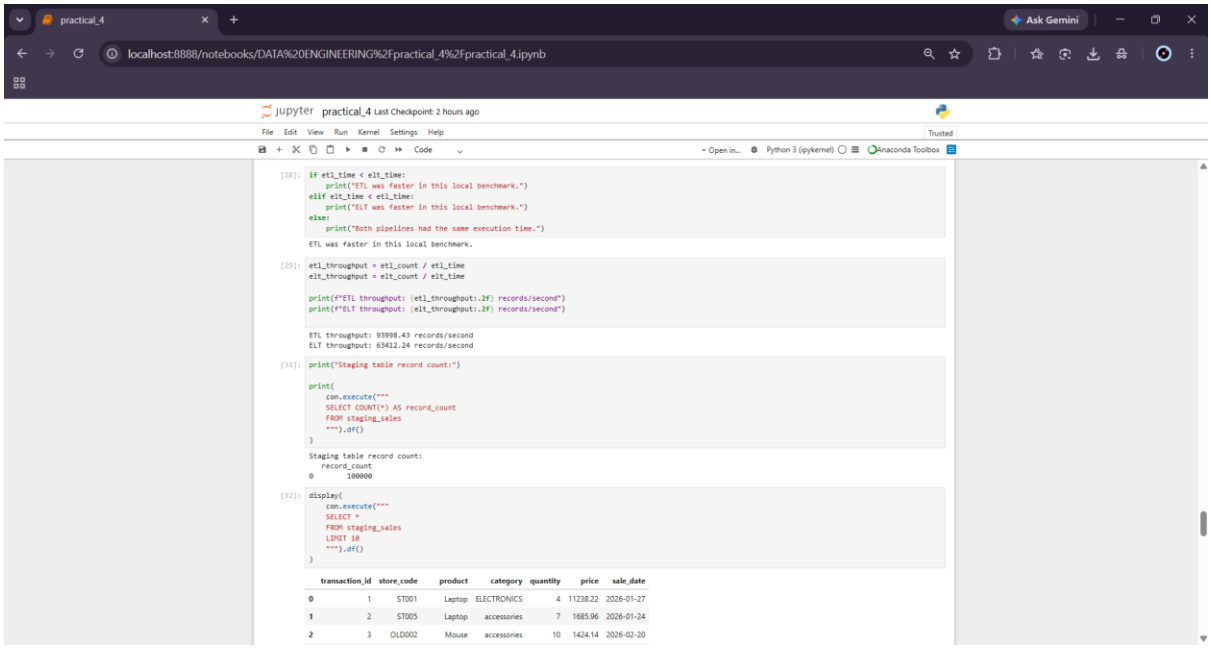

practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
[18]:
display(
con.execute("""
SELECT *
FROM elt_sales
LIMIT 10
""").df()
)
transaction_id store_code product category quantity price total_amount sale_date
0 1 ST001 Laptop Electronics 4 11238.22 44952.88 2026-01-27
1 2 ST005 Laptop Accessories 7 1685.96 11801.72 2026-01-24
2 3 ST002 Mouse Accessories 10 1424.14 14241.40 2026-02-20
3 4 ST005 Monitor Electronics 8 29504.36 236034.88 2026-01-02
4 5 ST002 USB Cable Accessories 6 13965.78 83794.68 2026-02-25
5 6 ST003 Laptop Electronics 7 4926.15 34483.05 2026-03-30
6 7 ST005 Keyboard Electronics 8 26857.78 214862.24 2026-04-07
7 8 ST001 Headphones Electronics 10 44284.04 442840.40 2026-04-03
8 9 ST005 Mouse Electronics 1 33097.04 33097.04 2026-03-16
9 10 ST001 Mouse Electronics 7 13970.88 97796.16 2026-06-12
[19]:
print("ELT Store Codes:")
print(
con.execute("""
SELECT DISTINCT store_code
FROM elt_sales
ORDER BY store_code
""").df()
)
print("\nELT Categories:")
print(
con.execute("""
SELECT DISTINCT category
FROM elt_sales
ORDER BY category

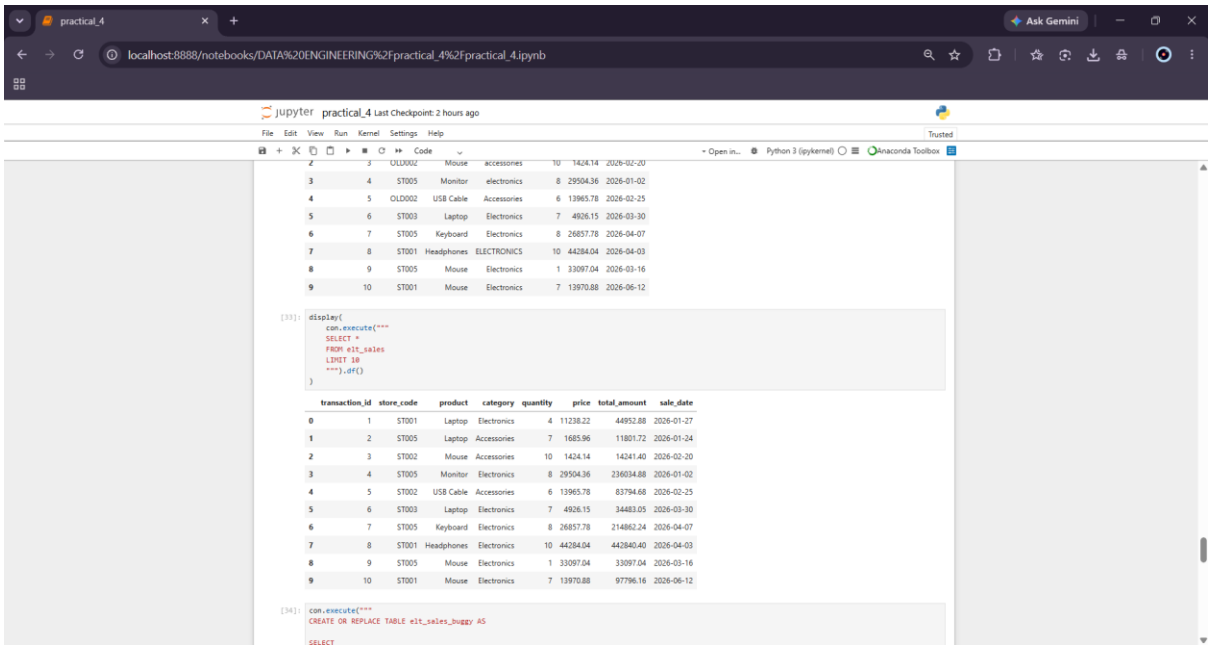

practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
FROM elt_sales
ORDER BY category
""").df()
)
ELT Store Codes:
store_code
0 ST001
1 ST002
2 ST003
3 ST004
4 ST005
ELT Categories:
category
0 Accessories
1 Electronics
[20]: elt_end = time.perf_counter()
elt_time = elt_end - elt_start
print(f"ELT execution time: {elt_time:.4f} seconds")
ELT execution time: 1.5770 seconds
[21]: etl_count = con.execute("""
SELECT COUNT(*)
FROM etl_sales
""").fetchone()[0]
elt_count = con.execute("""
SELECT COUNT(*)
FROM elt_sales
""").fetchone()[0]
print("ETL records:", etl_count)
print("ELT records:", elt_count)
ETL records: 100000
ELT records: 100000
[22]: etl_result = con.execute("""
SELECT *
FROM etl_sales
ORDER BY transaction_id
""").df()
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share

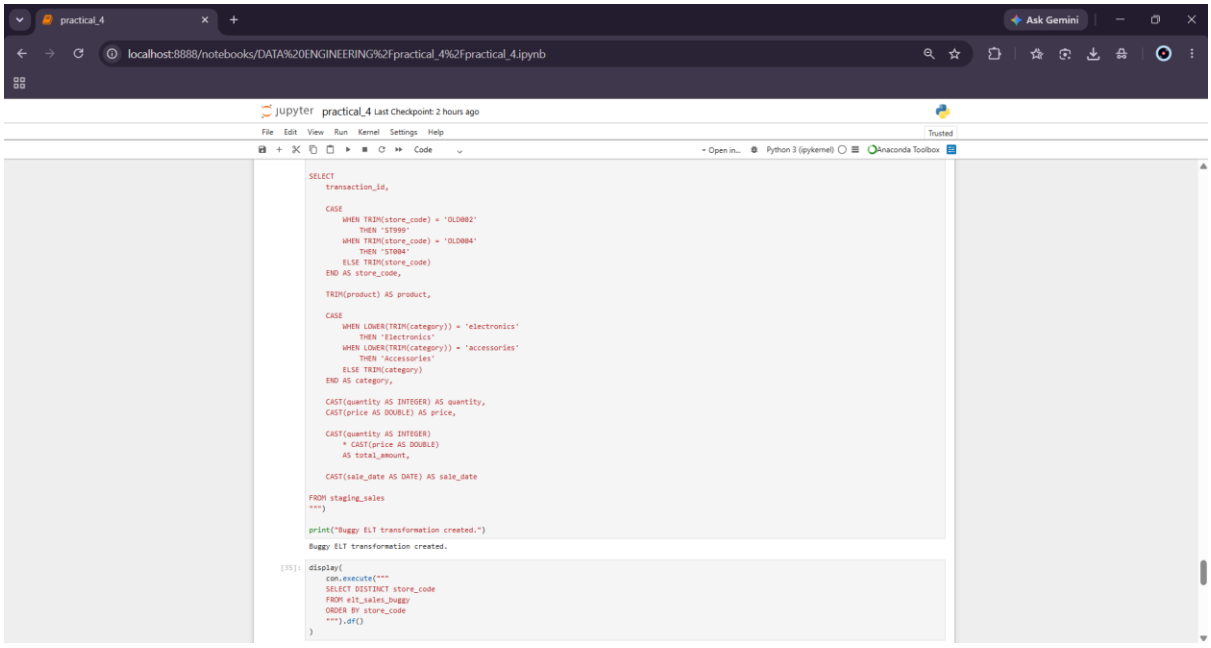
practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
""").df()
elt_result = con.execute("""
SELECT *
FROM elt_sales
ORDER BY transaction_id
""").df()
print("Same number of records:",
len(etl_result) == len(elt_result))
print("Same columns:",
list(etl_result.columns) == list(elt_result.columns))
Same number of records: True
Same columns: False
[23]: print("ETL data types:")
print(etl_result.dtypes)
print("\nELT data types:")
print(elt_result.dtypes)
ETL data types:
transaction_id int64
store_code object
product object
category object
quantity int64
price float64
sale_date datetime64[ns]
total_amount float64
dtype: object
ELT data types:
transaction_id int64
store_code object
product object
category object
quantity int32
price float64
total_amount float64
sale_date datetime64[us]
dtype: object
[24]: for column in etl_result.columns:
same = etl_result[column].astype(str).equals(
elt_result[column].astype(str)

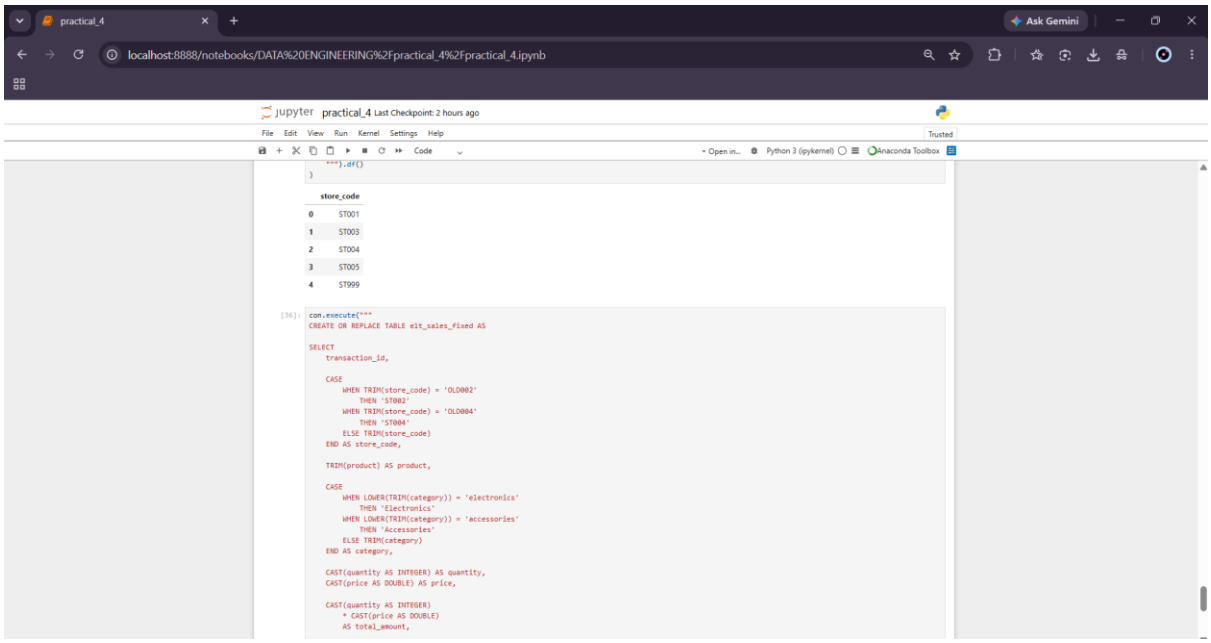

practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
same = etl_result[column].astype(str).equals(
elt_result[column].astype(str)
)
print(column, ":", same)
transaction_id : True
store_code : True
product : True
category : True
quantity : True
price : True
sale_date : True
total_amount : True
[25]:
Make copies
etl_final = etl_result.copy()
elt_final = elt_result.copy()
Use the same column order for both DataFrames
common_columns = sorted(
set(etl_final.columns) & set(elt_final.columns)
)
etl_final = etl_final[common_columns]
elt_final = elt_final[common_columns]
Reset row indexes
etl_final = etl_final.reset_index(drop=True)
elt_final = elt_final.reset_index(drop=True)
Convert values to strings for logical comparison
etl_final = etl_final.astype(str)
elt_final = elt_final.astype(str)
print("ETL columns:")
print(list(etl_final.columns))
print("\nELT columns:")
print(list(elt_final.columns))
print("\nSame number of records:",
len(etl_final) == len(elt_final))
print("Same columns:",
list(etl_final.columns) == list(elt_final.columns))

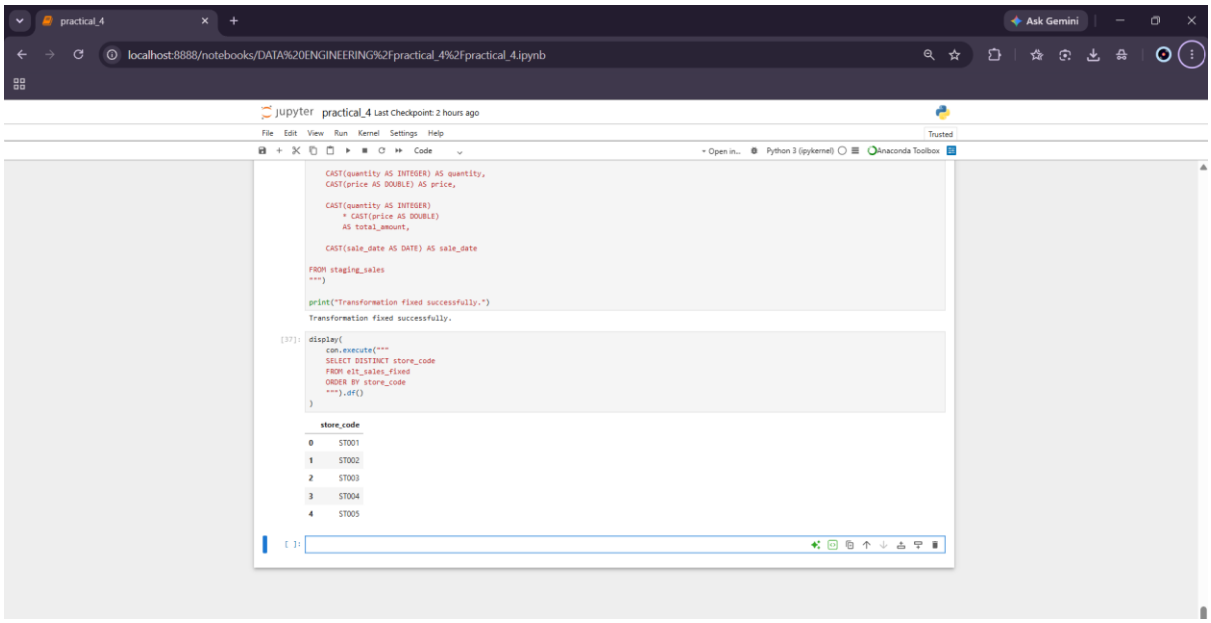

practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
print("Same columns:",
list(etl_final.columns) == list(elt_final.columns))
print("ETL and ELT data values identical:",
etl_final.equals(elt_final))
ETL columns:
['category', 'price', 'product', 'quantity', 'sale_date', 'store_code', 'total_amount', 'transaction_id']
ELT columns:
['category', 'price', 'product', 'quantity', 'sale_date', 'store_code', 'total_amount', 'transaction_id']
Same number of records: True
Same columns: True
ETL and ELT data values identical: True
[26]:
performance = pd.DataFrame({
"Pipeline": ["ETL", "ELT"],
"Execution Time (seconds)": [
etl_time,
elt_time
],
"Records Processed": [
etl_count,
elt_count
]
})
display(performance)
Pipeline Execution Time (seconds) Records Processed
0 ETL 1.063848 100000
1 ELT 1.576983 100000
[27]:
difference = abs(etl_time - elt_time)
print(f"ETL execution time: {etl_time:.4f} seconds")
print(f"ELT execution time: {elt_time:.4f} seconds")
print(f"Difference: {difference:.4f} seconds")
ETL execution time: 1.0638 seconds
ELT execution time: 1.5770 seconds
Difference: 0.5131 seconds
[28]:
if etl_time < elt_time:


practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
[28]:
if etl_time < elt_time:
print("ETL was faster in this local benchmark.")
elif elt_time < etl_time:
print("ELT was faster in this local benchmark.")
else:
print("Both pipelines had the same execution time.")
ETL was faster in this local benchmark.
[29]:
etl_throughput = etl_count / etl_time
elt_throughput = elt_count / elt_time
print(f"ETL throughput: {etl_throughput:.2f} records/second")
print(f"ELT throughput: {elt_throughput:.2f} records/second")
ETL throughput: 93998.43 records/second
ELT throughput: 63412.24 records/second
[31]:
print("Staging table record count:")
print(
con.execute("""
SELECT COUNT(*) AS record_count
FROM staging_sales
""").df()
)
Staging table record count:
record_count
0 100000
[32]:
display(
con.execute("""
SELECT *
FROM staging_sales
LIMIT 10
""").df()
)
transaction_id store_code product category quantity price sale_date
0 1 ST001 Laptop ELECTRONICS 4 11238.22 2026-01-27
1 2 ST005 Laptop accessories 7 1685.96 2026-01-24
2 3 OLD002 Mouse accessories 10 1424.14 2026-02-20


practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
3 4 ST005 Monitor electronics 8 29504.36 2026-01-02
4 5 OLD002 USB Cable Accessories 6 13965.78 2026-02-25
5 6 ST003 Laptop Electronics 7 4926.15 2026-03-30
6 7 ST005 Keyboard Electronics 8 26857.78 2026-04-07
7 8 ST001 Headphones ELECTRONICS 10 44284.04 2026-04-03
8 9 ST005 Mouse Electronics 1 33097.04 2026-03-16
9 10 ST001 Mouse Electronics 7 13970.88 2026-06-12
[33]:
display(
con.execute("""
SELECT *
FROM elt_sales
LIMIT 10
""").df()
)
transaction_id store_code product category quantity price total_amount sale_date
0 1 ST001 Laptop Electronics 4 11238.22 44952.88 2026-01-27
1 2 ST005 Laptop Accessories 7 1685.96 11801.72 2026-01-24
2 3 ST002 Mouse Accessories 10 1424.14 14241.40 2026-02-20
3 4 ST005 Monitor Electronics 8 29504.36 236034.88 2026-01-02
4 5 ST002 USB Cable Accessories 6 13965.78 83794.68 2026-02-25
5 6 ST003 Laptop Electronics 7 4926.15 34483.05 2026-03-30
6 7 ST005 Keyboard Electronics 8 26857.78 214862.24 2026-04-07
7 8 ST001 Headphones Electronics 10 44284.04 442840.40 2026-04-03
8 9 ST005 Mouse Electronics 1 33097.04 33097.04 2026-03-16
9 10 ST001 Mouse Electronics 7 13970.88 97796.16 2026-06-12
[34]:
con.execute("""
CREATE OR REPLACE TABLE elt_sales_buggy AS
SELECT


practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
SELECT
transaction_id,
CASE
WHEN TRIM(store_code) = 'OLD002'
THEN 'ST999'
WHEN TRIM(store_code) = 'OLD004'
THEN 'ST004'
ELSE TRIM(store_code)
END AS store_code,
TRIM(product) AS product,
CASE
WHEN LOWER(TRIM(category)) = 'electronics'
THEN 'Electronics'
WHEN LOWER(TRIM(category)) = 'accessories'
THEN 'Accessories'
ELSE TRIM(category)
END AS category,
CAST(quantity AS INTEGER) AS quantity,
CAST(price AS DOUBLE) AS price,
CAST(quantity AS INTEGER)
* CAST(price AS DOUBLE)
AS total_amount,
CAST(sale_date AS DATE) AS sale_date
FROM staging_sales
""")
print("Buggy ELT transformation created.")
Buggy ELT transformation created.
[35]:
display(
con.execute("""
SELECT DISTINCT store_code
FROM elt_sales_buggy
ORDER BY store_code
""").df()
)


practical_4
Ask Gemini
localhost:8888/notebooks/DATA%20ENGINEERING%2Fpractical_4%2Fpractical_4.ipynb
jupyter practical_4 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
Anaconda Toolbox
""").df()
)
store_code
0 ST001
1 ST003
2 ST004
3 ST005
4 ST999
[36]:
con.execute("""
CREATE OR REPLACE TABLE elt_sales_fixed AS
SELECT
transaction_id,
CASE
WHEN TRIM(store_code) = 'OLD002'
THEN 'ST002'
WHEN TRIM(store_code) = 'OLD004'
THEN 'ST004'
ELSE TRIM(store_code)
END AS store_code,
TRIM(product) AS product,
CASE
WHEN LOWER(TRIM(category)) = 'electronics'
THEN 'Electronics'
WHEN LOWER(TRIM(category)) = 'accessories'
THEN 'Accessories'
ELSE TRIM(category)
END AS category,
CAST(quantity AS INTEGER) AS quantity,
CAST(price AS DOUBLE) AS price,
CAST(quantity AS INTEGER)
* CAST(price AS DOUBLE)
AS total_amount,



Key Questions / Analysis / Interpretation

Que 1: How does the separation of compute and storage in modern cloud data lakes shift the architectural preference from classic ETL models toward ELT?

Modern cloud data platforms commonly separate data storage from compute resources. Raw data can therefore be stored centrally and processed using scalable compute resources when required.

In ETL, transformations are performed before loading the data into the target system. This requires application-side processing and may require movement of raw data to application compute resources.

In ELT, raw data is loaded first and transformations are performed inside the target database or warehouse. This allows the database engine to use its own query-processing and compute capabilities.

Therefore, separation of storage and compute can make ELT suitable for large-scale analytical workloads because raw data can be retained and transformation processing can be performed using scalable database compute.

Que 2: Compare the pipelines in terms of maintaining robust data lineage tracking and recovering from intermediate transformation bugs.

ETL performs transformations in application memory before loading the final result. If intermediate datasets are not explicitly saved, debugging a transformation problem may require the pipeline to be executed again.

ELT provides a persistent staging layer between raw and transformed data:

Raw Data

↓

Staging

↓

Transformation

↓

Final Data

This makes it easier to inspect the original data and correct SQL transformation logic.

The experiment demonstrated this by intentionally introducing an incorrect store-code transformation and subsequently correcting it using the original staging_sales table.

Que 3: Evaluate how network resource costs shift when running transformation calculations inside application compute nodes versus inside database storage engines.

In an ETL architecture, raw data may need to move from storage to application compute nodes. After transformation, processed data may then be transferred to the target database.

In an ELT architecture, the raw data is first loaded into the database or warehouse and transformations are performed by the database engine.

Therefore, ELT can reduce unnecessary movement of large datasets between separate application and database environments when the database compute engine is located close to the stored data.

In this local experiment, network cost is minimal because all components operate on the same machine.

Final Analysis / Interpretation

The experiment successfully implemented and evaluated two data integration architectures: an ETL pipeline using Python and Pandas and an ELT pipeline using DuckDB SQL.

The ETL pipeline transformed the raw retail transaction data in application memory before loading the cleaned data into DuckDB. The ELT pipeline first loaded the raw records into a DuckDB staging table and then performed transformations using SQL.

Both pipelines processed the same dataset and produced logically equivalent cleaned results. Execution time was measured using the same local environment to provide a performance comparison.

The experiment also demonstrated differences in data lineage and transformation recovery. The ELT architecture retained the original data in the staging table, allowing transformation logic to be modified and executed again without recreating the raw input dataset.

The performance results obtained from the local benchmark should be interpreted according to the hardware, dataset size, implementation, and database configuration used during the experiment.

Final Conclusion

The practical successfully demonstrated the implementation and comparison of ETL and ELT data integration architectures using Python, Pandas, and DuckDB.

The ETL pipeline performed transformations in application memory before loading the processed data into DuckDB. The ELT pipeline loaded raw data into a staging table and performed transformations using the DuckDB analytical engine.

The experiment demonstrated that both architectures can produce equivalent cleaned datasets while differing in transformation location, data lineage, debugging, resource utilization, and maintenance requirements.

The ELT architecture provides a persistent staging layer that supports easier inspection and recovery from transformation errors. ETL provides a straightforward approach when transformation logic is naturally implemented in application code.

The appropriate architecture depends on factors such as dataset volume, infrastructure capabilities, compute location, network costs, transformation complexity, lineage requirements, and maintenance needs.

GITHUB REPOSITORY

https://github.com/tarjani-patel/D25DIT091_DataEngineering_Practicals/tree/main/practical4